GameSearch

Moteur de recherche de jeux vidéos pour les fans de Gaming

Document de conception

1. Présentation du sujet

1.1 Contexte et description du projet

Le marché mondial du jeu vidéo représente aujourd'hui l'une des industries culturelles les plus dynamiques, avec des milliers de nouveaux jeux publiés chaque année sur différentes plateformes.
Les joueurs doivent consulter de multiples sites pour comparer les jeux, vérifier leur popularité, leur qualité ou leur disponibilité.
Parallèlement, de nombreux studios indépendants rencontrent des difficultés à rendre leurs jeux visibles auprès du public, faute de plateformes accessibles permettant de les présenter efficacement.

Dans ce contexte, GameSearch a été mise sur pied. C'est une plateforme web de référencement et de recherche de jeux vidéo destinée aux fans de gaming. Elle permet à tout utilisateur de découvrir, rechercher, filtrer et consulter des jeux vidéo selon de nombreux critères (genre, plateforme, année, note, tags) à travers une interface moderne et réactive.

La plateforme offre également aux partenaires (éditeurs, distributeurs, data providers) un canal privilégié pour alimenter le catalogue via une API dédiée et des événements asynchrones Kafka, garantissant une ingestion de données rapide, fiable et découplée.

1.2 Valeur ajoutée

GameSearch se distingue par :

- Une recherche puissante et multicritères : titre, genre, plateforme, année, note, éditeur, tags
- Un catalogue enrichi alimenté en temps réel par des partenaires via Kafka (flux asynchrones)
- Une expérience utilisateur centrée gaming : fiches jeux détaillées, recommandations similaires
- Un système de favoris et d'alertes personnalisées (nouveautés, sorties à venir)
- Une architecture distribuée robuste et scalable (Spring Boot + Kafka + Docker)

2. Profils utilisateurs

Profil 1 : Le Gamer Passionné (Visiteur/ Utilisateur connecté)

Attribut	Détail
Nom personna	Alex, 22 ans, étudiant
Objectif principal	Trouver rapidement un jeu à jouer selon ses goûts
Comportements	Recherche par genre, lit les fiches détaillées, compare les jeux
Frustrations	Trop de résultats non pertinents, manque de filtres combinés
Valeur attendue	Recherche rapide, fiches complètes, recommandations personnalisées

Profil 2 : Le Partenaire ou Éditeur

Attribut	Détail
Nom personna	Game Corp, editeur / distributeur de jeux

Objectif principal	Référencer et mettre ses jeux dans le catalogue
Comportements	Utilise l’api ou publie des messages kafka pour alimenter le catalogue
Frustrations	Intégration complexe, latence d’indexation, manque de feedback
Valeur attendue	API simple, Ingestion asynchrone (kafka), retour d'état en temps réel

Profil 3 : L’administrateur de la plateforme

Attribut	Détail
Nom personna	Sam, administrateur technique
Objectif principal	Gérer le catalogue, superviser la plateforme, modérer les contenus
Comportements	Créer/modifier/supprimer des jeux, gère les partenaires, surveiller les logs
Frustrations	Donnees incompletes et erreurs d’ingestion non loguées
Valeur attendue	Interface admin fiable, logs exploitables, gestions des partenaires

3. Users Stories

3.1 : Le Gamer Passionné (Visiteur non connecté)

ID	En tant que…	Je veux…	Afin de…	Priorité
US-G01	Visiteur	Rechercher un jeu par titre	Trouver rapidement un jeu que je connais	🔴 Must
US-G02	Visiteur	Consulter la fiche détaillée d'un jeu	Voir la description, les screenshots, les notes	🔴 Must
US-G03	Visiteur	Filtrer les jeux par genre , plateforme, année	Affiner ma recherche selon mes goûts	🔴 Must
US-G04	Visiteur	Voir la liste des jeux récents et/ou populaires	Decouvrir des nouveautes dans chercher	🟡 Should
US-G05	Visiteur	Accéder à la plateforme sans inscriptions	Tester sans engagement	🟡 Should
US-G06	Visiteur	Voir les jeux similaires sur la fiche d'un jeu	Obtenir des recommandations pertinentes	🔴 Must

3.2 : Le Gamer Passionné (Utilisateur connecté)

ID	En tant que…	Je veux…	Afin de…	Priorité
US-U01	Utilisateur connecté	Créer un compte et me connecter	Accéder à mes fonctionnalites personnelles	🟡 Should
US-U02	Utilisateur connecté	Ajouter un jeu à mes favoris	Retrouver facilement mes jeux préférés	🟡 Should
US-U03	Utilisateur connecté	Recevoir une alerte sur une nouvelle sortie	Etre notifié des jeux attendus	🟢 Could
US-U04	Utilisateur connecté	Gérer ma liste de favoris	Ajouter/supprimer des entrées	🟡 Should

3.3 : Partenaire / Éditeur

ID	En tant que…	Je veux…	Afin de…	Priorité
US-P01	Partenaire API	Soumettre un jeu via l'API REST	Référencer mon jeu sur la plateforme	🔴 Must
US-P02	Partenaire API	Publier un événement Kafka pour ajouter un jeu	Intégrer mon système d'édition en asynchrone	🔴 Must
US-P03	Partenaire API	Mettre à jour les informations d'un jeu existant	Corriger ou enrichir une fiche	🔴 Must
US-P04	Partenaire API	M'authentifier via une clé API populaires	Sécuriser mes accès d'écriture	🔴 Must

US-P05	Partenaire API	Recevoir un retour d'état après ingestion	Savoir si mon jeu a été correctement indexé	🟡 Should
US-P06	Partenaire API	Importer un catalogue en masse	Alimenter rapidement le catalogue	🟢 Could

4. Architecture Technique

4.1 : Vue d'ensemble

L'architecture suit un découpage en couches classiques (Controller -> Service -> Repository) avec une couche événementielle Kafka pour les flux asynchrones d'ingestion partenaires.

Composant	**Technologie**	**Rôle**
API Backend	Java 21 + Spring Boot 3	Exposition des endpoints REST, logique métier
Messaging	Apache Kafka	Ingestion asynchrone des jeux par les partenaires
Base de données	PostgreSQL	Persistance du catalogue de jeux
Frontend	React	Interface utilisateur MVP (recherche, fiches, favoris)
Conteneurisation	Docker + docker-compose	Stack complète reproductible
CI/CD	Gitlab CI/CD	Build, tests, coverage, déploiement
Monitoring	Spring Actuator	Endpoints /health, logs

		structurés
Documentation API	Swagger / OPenAPI	Documentation interactive des endpoints

4.2 : Flux Kafka

Deux topics kafka principaux sont identifiés pour le MVP:

- game-ingestion-topic : publié par les partenaires (via API ou directement), consomme par le service d'indexation pour créer/mettre à jour une fiche jeu.
- game-ingestion-status-topic: publié par le service d'indexation en retour, consommé par le partenaire pour connaître le statut de son ingestion (succès, erreur, données manquantes)

Structure d'un message Kafka (game-ingestion-topic):

Champ	**Type**	**Obligatoire**	**Description**
gameId	String(UUID)	Non	ID externe partenaire
title	String	Oui	Titre du jeu
genre	List<String>	Oui	Actions, RPG, …
platform	List<String>	Oui	PC, PS5, …
releaseYear	Integer	Oui	Année de sortie
publisher	String	Non	Editeur / Studio
description	String	Non	Description longue
coverUrl	String	Non	URL de la jaquette

rating	Float	Non	Note (0.0 - 10.0)
tags	List<String>	Non	Tags libres (eg: ‘open-world)

5. Modele de Donnees

5.1 : Entité principale : Game

Champ	Type SQL	Contraintes	Description
id	UUID	PK, NOT NULL	ID interne
external_id	VARCHAR(255)	UNIQUE, NULLABLE	ID partenaire externe
title	VARCHAR(255)	NOT NULL	Titre du jeu
update_at	TIMESTAMP	NOT NULL	Dernière modification
release_year	INTEGER	NOT NULL	Année de sortie
publisher	VARCHAR(255)	NULLABLE	Editeur / Studio
description	TEXT	NULLABLE	Description longue
cover_url	VARCHAR(255)	NULLABLE	URL de la jaquette
rating	FLOAT	CHECK 0.0-10.0	Note globale
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Date de création

5.2 : Entités liées

Entité	Champs principaux	Relations
Genre	id, name	Many-to-Many avec Game
Platform	id, name	Many-to-Many avec Game
Tag	id, name	Many-to-Many avec Game
Partner	id, name, api_key_hash, active	One-to-Many avec Game (Source)

6. Sécurité

Conformément aux exigences du projet, la sécurité est géré à deux niveaux :

Périmètre	Mécanisme	Détail
Endpoints lecture (GET)	Public, sans auth	Accessible à tous, séparation visible des routes lectures/écriture
Endpoints écriture (POST/PUT/DELETE admin)	Spring security + JWT ou Session	Accès réservé aux administrateurs authentifiés
Endpoints partenaire (POST ingestion)	Cle API (header X-API-Key)	Chaque partenaire a une clé unique hachée en base
Validation entrées	Jakarta validation sur les DTOs	Contraintes @NotNull, @Size, @Min, @Max, …

		sur tous les DTOs
Gestion des erreurs	GlobalExceptionHanlder(@ControllerAdvice)	Codes HTTP cohérents, messages bien structurés

7. Endpoints API REST (MVP)

Méthode	**Route**	**Auth**	**Description**
GET	/games	Public	Liste paginée de jeux avec filtres
GET	/games/{id}	Public	Détail d'un jeu
GET	/games/search?=q &genre=&platform =&year=	Public	Recherche multicritères
POST	/games	Admin JWT	Créer un jeu manuellement
PUT	/games{id}	Admin JWT	Modifier un jeu
DELETE	/games{id}	Admin JWT	Supprimer un jeu
POST	/partner/games	API Key	Soumettre un jeu (partenaire)
POST	/partner/games/bulk	API Key	import de masse (CSV/JSON)
GET	/genres	Public	Liste des genres disponibles
GET	/platforms	Public	Liste des plateformes disponibles
GET	/actuator/health	Public	Monitoring - etat de la plateforme

8. Tests

Type de Test	Outil	Périmètre	Objectif de couverture
Tests unitaires	JUnit + Mockito	Couche Service, utilitaires, mapping DTO	>= 80% du code métier
Test d'intégration	Spring Boot Test + JTestcontainers (PostgreSQL, Kafka)	Repositories, API endpoints, flux Kafka	>= 70% global
Tests e2e / scenarios	Postman Collection	Parcours utilisateur complet (recherche -> fiche)	Scénarios critiques couverts
Audit sécurité	Agent IA “Expert sécurité senior” ou autre …	Endpoints, auth, validation, erreurs HTTP	Rapport d’audit

9. Stack Devops

Element	Outil	Description
Versionning	Git + GitLab	main + branches feature (feature/US-XXX), MR obligatoire
CI Pipeline	GitLab CI/CD	Build Maven -> Tests -> Coverage (JaCoCo) -> Docker build
Containerisation	Docker + docker-compose	Services : backend, frontend, PostgresSQL, Kafka, Zookeeper
Artifact repository	GitLab Container Registry / Docker Hub	Images Docker versionnées pour la release 1.0
Monitoring	Spring Actuator	/health, /info, logs JSON exploitables
Documentation	Swagger UI (SpringDoc)	Interfaces interactive des endpoints